

What we actually need to build.....	2
The actual CTF challenges.....	2
Web.....	2
Cryptography.....	3
Awareness.....	3
Ethics.....	3
Flag system.....	3
Hints and explanations.....	4
Why this matters:.....	4
What our group would actually code.....	4
1. The CTF platform.....	4
2. The challenges.....	4
And then the report.....	4
Phase I.....	5
Phase II.....	5
Phase III.....	5
Phase IV.....	5

Team Members:

LEEN HUSSAMLADEEN ADEL NASSER

ALJAWHARA MOHMMAD NASSER ALRUZUQ

AREEJ AADEL SALEM ALMARI

SHOUG FAWAZ ABDULLAH ALOMRAN

If you pick **#28 Mini Capture-The-Flag (CTF) Security Lab Platform**, you're essentially building your own **small CTF website** where users solve cybersecurity challenges, submit flags, and have their progress tracked. The instructor's description is fairly specific about the required functionality.

What we actually need to build

Think of the basic flow as:

The user opens your CTF → chooses a challenge → solves it → finds a flag → submits the flag → system validates it → challenge is marked complete → progress/score updates.

The project brief requires a **web or desktop interface**, multiple challenge categories, **flag validation**, **progress tracking**, explanations/hints, and a **scoreboard or completion status**.

I would make it a web application. A sensible structure would be:

Page	What it does
Home	Introduces the CTF/security lab
Login/Register	Identifies each participant
Dashboard	Shows progress, points and completed challenges
Challenges	Lists all available challenges by category
Challenge Details	Scenario, instructions, downloadable/material resources, hint and flag submission
Scoreboard	Shows scores/completion
Profile/Progress	Shows that users solved challenges

The actual CTF challenges

The assignment explicitly suggests **Web, Crypto, Awareness, and Ethics** as categories.

You don't need to build Hack The Box. A small controlled lab is enough. For example, you could create something like:

Web

- Inspect the page source to discover a hidden flag.

- Find information exposed in a deliberately insecure page.
- Identify an authentication/configuration weakness in a controlled demo.

Cryptography

- Decode Base64.
- Solve a Caesar cipher.
- Perform simple frequency analysis.
- Identify/decrypt a basic encoded message.

Awareness

- Examine a fake phishing email and identify the suspicious element.
- Decide which URL is legitimate.
- Identify a social-engineering technique.

Ethics

- Present cybersecurity scenarios and ask what action is permitted/ethical.
- Explain why the answer is appropriate after submission.

This actually connects nicely to CYS401 because your course covers things such as vulnerabilities, threats, phishing/social engineering, confidentiality, integrity, availability, authentication and security controls. [chapter-2-security-foundations-and-principles.pdf](#)

Flag system

Each challenge would have something like:

`CYS401 {phishing_detected}`

The user shouldn't be able to simply see the correct answer in the normal interface. They solve the challenge and submit their discovered flag.

Your backend checks it:

Correct → mark challenge solved → award points → update progress.

Incorrect → tell them it's incorrect and allow another attempt.

You could also track the number of attempts.

Hints and explanations

The instructor specifically requires “**Hints or explanations to support learning.**”

So each challenge could have:

- **Hint 1:** small clue
- **Hint 2:** stronger clue

Then, after solving it:

Why this matters:

This challenge demonstrates how exposed information can violate confidentiality...

That would help you connect the implementation directly to course concepts such as the **CIA triad**. The course defines confidentiality as preventing unauthorized access, integrity as maintaining accurate/reliable information, and availability as ensuring authorized users can access information when required. [chapter-2-security-foundations-and-principles.pdf](#)

What our group would actually code

There are really **two pieces**.

1. The CTF platform

Frontend + backend/database for:

- accounts
- challenges
- categories
- flags
- submissions
- points
- progress
- scoreboard
- hints

2. The challenges

Your group creates the vulnerable/educational scenarios that users solve.

That’s important: simply making a website containing cybersecurity quiz questions wouldn’t fully represent the project description. The brief specifically says users should **analyze a security scenario, identify the security issue, and submit a correct flag.**

And then the report

You’re still responsible for the normal semester-project requirements.

Our **13–15 page report** needs to cover:

Phase I

Introduction/security analysis: what CTFs are, assets involved, risks, threats, attacks and vulnerabilities.

Phase II

Literature review: existing approaches/platforms and relevant security incidents or related implementations.

Phase III

Implementation: software/hardware requirements, how your CTF works, screenshots of the interface, explanations, architecture/implementation, and your source code in the appendix.

Phase IV

Application & recommendations: connect what you’ve learned in CYS401 to the project and provide security recommendations.

Then **Phase V** is the **10-minute presentation/demo + Q&A**.

During the demo, you could literally open your website, log in as a participant, select a challenge, solve it, submit `CYS401{...}`, show the successful validation, and then show the updated progress/scoreboard.

So the CTF option is **not “build a hacking website.”** It’s more specifically:

Build an educational cybersecurity platform containing several controlled security challenges that users solve to obtain flags, with automatic validation, progress tracking, hints/explanations, and a scoreboard/completion system.

And importantly, the assignment only specifies **multiple categories**, not a required number of challenges. So you have room to decide how ambitious you want the implementation to be.